

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Metody Systemowe i Decyzyjne

Projekt semestralny – materiały dodatkowe

Zastosowanie metaheurystyk w problemie harmonogramowania produkcji

1. Opis problemu decyzyjnego

Rozważany jest problem harmonogramowania produkcji typu **Permutation Flow Shop Scheduling Problem (PFSP)**.

Dany jest system produkcyjny składający się z:

- N zadań (jobs),
- M maszyn.

Każde zadanie musi zostać wykonane na wszystkich maszynach w tej samej kolejności technologicznej:

$M1 \rightarrow M2 \rightarrow \dots \rightarrow MM$

Nie dopuszcza się przerwania realizacji zadania na maszynie.

2. Dane wejściowe

Dane problemu określa macierz czasów przetwarzania

$P=[p_{j,m}]$

gdzie

- $p_{j,m}$ — czas przetwarzania zadania j na maszynie m

Macierz ma wymiar

$N \times M$

Przykład macierzy czasów

$P=546273635$

Zadanie M1 M2 M3

J1 5 2 6

Zadanie M1 M2 M3

J2	4	7	3
J3	6	3	5

3. Zmienne decyzyjne

Rozwiązanie problemu stanowi **permutacja zadań**

$$\pi = (\pi_1, \pi_2, \dots, \pi_N)$$

która określa kolejność realizacji zadań.

Interpretacja permutacji

Jeżeli

$$\pi = (3, 1, 2)$$

to zadania wykonywane będą w kolejności:

1. zadanie 3
2. zadanie 1
3. zadanie 2

4. Funkcja celu

Celem optymalizacji jest minimalizacja **czasu zakończenia wszystkich zadań (makespan)**

$C_{\max}$

czyli czasu zakończenia ostatniego zadania na ostatniej maszynie.

5. Obliczanie czasu realizacji harmonogramu

Czas zakończenia zadania na pozycji i harmonogramu na maszynie m oznaczamy jako

$$C(i, m)$$

Obliczamy go ze wzoru

$$C(i, m) = \max(C(i-1, m), C(i, m-1)) + p_{\pi(i), m}$$

6. Objasnienie oznaczeń

- i — pozycja zadania w harmonogramie
- m — numer maszyny
- $C(i, m)$ — czas zakończenia zadania

- π_i — numer zadania na pozycji i
- $p_{j,m}$ — czas przetwarzania zadania j na maszynie m

Zapis

$p_{\pi_i,m}$

oznacza czas przetwarzania **zadania znajdującego się na pozycji i** harmonogramu na maszynie m .

7. Wartość funkcji celu

Ostateczna wartość funkcji celu:

$C_{max}=C(N,M)$

8. Algorytmy optymalizacji

W projekcie należy zaimplementować następujące algorytmy metaheurystyczne:

1. Algorytm genetyczny (Genetic Algorithm)
2. Symulowane wyżarzanie (Simulated Annealing)
3. Algorytm kolonii mrówek (Ant Colony Optimization)
4. Algorytm pszczoły (Bees Algorithm)

9. Algorytm symulowanego wyżarzania

Algorytm **Simulated Annealing (SA)** polega na iteracyjnym przeszukiwaniu przestrzeni rozwiązań.

W każdej iteracji:

1. generowane jest nowe rozwiązanie,
2. obliczana jest funkcja celu,
3. podejmowana jest decyzja o jego akceptacji.

10. Dlaczego dopuszczamy gorsze rozwiązania

W wielu problemach optymalizacji istnieje wiele **minimów lokalnych**.

Algorytm, który akceptuje wyłącznie rozwiązania lepsze, może zatrzymać się w minimum lokalnym.

Dlatego w algorytmie symulowanego wyżarzania dopuszcza się **czasami akceptację rozwiązań gorszych**, aby umożliwić dalsze przeszukiwanie przestrzeni rozwiązań.

11. Kryterium akceptacji rozwiązania

Niech

- $x_{current}$ — aktualne rozwiązanie
- x_{new} — nowe rozwiązanie

Funkcja celu

$$f(x) = C_{max}$$

Zmiana funkcji celu

$$\Delta = f(x_{new}) - f(x_{current})$$

czyli

$$\Delta = C_{max_{new}} - C_{max_{current}}$$

12. Interpretacja wartości Δ

Jeżeli

$$\Delta < 0$$

nowe rozwiązanie jest lepsze i zostaje zaakceptowane.

Jeżeli

$$\Delta > 0$$

rozwiązanie jest gorsze i może zostać zaakceptowane z pewnym prawdopodobieństwem.

13. Prawdopodobieństwo akceptacji

$$P = e^{-T\Delta}$$

gdzie

- T — temperatura algorytmu.

14. Implementacja w MATLAB

```
delta = newCost - currentCost;
if delta <= 0
    accept = true;
else
    P = exp(-delta/T);
    accept = rand < P;
end
```

15. Generowanie sąsiedztwa dla permutacji

W algorytmach optymalizacyjnych konieczne jest generowanie nowych rozwiązań w pobliżu aktualnego rozwiązania.

Typowe operatory dla permutacji:

Swap

zamiana dwóch zadań

[1 2 3 4 5]

→

[1 4 3 2 5]

Insert

przeniesienie elementu w inne miejsce

[1 2 3 4 5]

→

[1 3 4 2 5]

Inversion

odwrócenie fragmentu permutacji

[1 2 3 4 5]

→

[1 4 3 2 5]

16. Sugestie implementacyjne — Algorytm genetyczny

Reprezentacja rozwiązania:

- pojedynczy osobnik = permutacja zadań

Przykład:

```
perm = randperm(N);
```

Ocena osobnika

Każdy osobnik oceniany jest za pomocą funkcji celu `makespan(P, perm)`.

```
cost = makespan(P, perm);
```

```
fitness = 1 / cost;
```

Przykład mutacji typu swap

```
function child = mutate_swap(child)
n = length(child);
i = randi(n);
j = randi(n);
while j == i
    j = randi(n);
end
child([i j]) = child([j i]);
end
```

Przykład selekcji turniejowej

```
function selected = tournament_selection(population, costs, tournamentSize)
```

```

popSize = size(population, 1);
idx = randperm(popSize, tournamentSize);
bestIdx = idx(1);
for k = 2:tournamentSize
    if costs(idx(k)) < costs(bestIdx)
        bestIdx = idx(k);
    end
end
selected = population(bestIdx, :);
end

```

Przykład prostego krzyżowania OX (Order Crossover)

```

function child = ox_crossover(parent1, parent2)
n = length(parent1);
child = zeros(1, n);
a = randi(n);
b = randi(n);
if a > b
    temp = a;
    a = b;
    b = temp;
end
child(a:b) = parent1(a:b);
remaining = parent2(~ismember(parent2, child));
pos = [1:a-1, b+1:n];
child(pos) = remaining;
end

```

Schemat iteracji GA

```

for gen = 1:maxGenerations

    for i = 1:popSize
        costs(i) = makespan(P, population(i,:));
    end

    newPopulation = zeros(size(population));

    for i = 1:2:popSize

        parent1 = tournament_selection(population, costs, 3);
        parent2 = tournament_selection(population, costs, 3);

        child1 = ox_crossover(parent1, parent2);
        child2 = ox_crossover(parent2, parent1);

        if rand < mutationRate
            child1 = mutate_swap(child1);
        end

        if rand < mutationRate

```

```

        child2 = mutate_swap(child2);
    end

    newPopulation(i,:) = child1;
    if i+1 <= popSize
        newPopulation(i+1,:) = child2;
    end
end

population = newPopulation;

end

```

17. Sugestie implementacyjne — Ant Colony Optimization

W algorytmie mrówkowym każda mrówka buduje permutację krok po kroku.

Inicjalizacja feromonów

```
tau = ones(N, N);
```

Można przyjąć, że $\tau(i, j)$ oznacza atrakcyjność umieszczenia zadania j po zadaniu i .

Heurystyka

Prosta heurystyka może być oparta na sumarycznym czasie przetwarzania zadania:

```
jobTime = sum(P, 2);
eta = 1 ./ jobTime;
```

Wybór kolejnego zadania

```

function nextJob = select_next_job(availableJobs, tauRow, eta, alpha, beta)
prob = zeros(1, length(availableJobs));
for k = 1:length(availableJobs)
    j = availableJobs(k);
    prob(k) = (tauRow(j)^alpha) * (eta(j)^beta);
end
prob = prob / sum(prob);
r = rand;
cumProb = cumsum(prob);
nextJob = availableJobs(find(r <= cumProb, 1, 'first'));
end

```

Budowa pojedynczego rozwiązania przez mrówkę

```

function perm = build_ant_solution(P, tau, eta, alpha, beta)
N = size(P,1);
perm = zeros(1,N);
availableJobs = 1:N;
firstIdx = randi(N);
perm(1) = availableJobs(firstIdx);

```

```

availableJobs(firstIdx) = [];
for pos = 2:N
    prevJob = perm(pos-1);

    nextJob = select_next_job(availableJobs, tau(prevJob,:), eta, alpha, beta);

    perm(pos) = nextJob;
    availableJobs(availableJobs == nextJob) = [];
end
end

```

Aktualizacja feromonów

```

tau = (1 - rho) * tau;
for ant = 1:numAnts
    perm = antSolutions(ant,:);
    cost = antCosts(ant);

    for k = 1:N-1
        i = perm(k);
        j = perm(k+1);
        tau(i,j) = tau(i,j) + 1 / cost;
    end
end

```

Schemat iteracji ACO

```

for iter = 1:maxIters

    for ant = 1:numAnts
        antSolutions(ant,:) = build_ant_solution(P, tau, eta, alpha, beta);
        antCosts(ant) = makespan(P, antSolutions(ant,:));
    end

    tau = (1 - rho) * tau;

    for ant = 1:numAnts
        perm = antSolutions(ant,:);
        cost = antCosts(ant);

        for k = 1:N-1
            i = perm(k);
            j = perm(k+1);
            tau(i,j) = tau(i,j) + 1 / cost;
        end
    end
end

```

18. Sugestie implementacyjne — Bees Algorithm

W algorytmie pszczelim część rozwiązań generowana jest losowo, a część powstaje przez lokalne przeszukiwanie najlepszych obszarów.

Losowe rozwiązania początkowe

```
for i = 1:numBees
    population(i,:) = randperm(N);
    costs(i) = makespan(P, population(i,:));
end
```

Lokalna modyfikacja rozwiązania

Najprostszy operator: swap.

```
function newPerm = local_search_swap(perm)
newPerm = perm;
n = length(perm);
i = randi(n);
j = randi(n);
while j == i
    j = randi(n);
end
newPerm([i j]) = newPerm([j i]);
end
```

Przeszukiwanie sąsiedztwa najlepszego rozwiązania

```
function bestNeighbor = explore_neighborhood(P, perm, nSearch)
bestNeighbor = perm;
bestCost = makespan(P, perm);
for k = 1:nSearch
    candidate = local_search_swap(perm);
    candidateCost = makespan(P, candidate);

    if candidateCost < bestCost
        bestNeighbor = candidate;
        bestCost = candidateCost;
    end
end
end
```

Schemat iteracji Bees Algorithm

```
for iter = 1:maxIters

    for i = 1:numBees
        costs(i) = makespan(P, population(i,:));
    end

    [costs, order] = sort(costs);
    population = population(order,:);

    newPopulation = population;
```

```

    for i = 1:numEliteSites
        newPopulation(i,:) = explore_neighborhood(P, population(i,:), elite-
SearchSize);
    end

    for i = numEliteSites+1:numSelectedSites
        newPopulation(i,:) = explore_neighborhood(P, population(i,:), selected-
SearchSize);
    end

    for i = numSelectedSites+1:numBees
        newPopulation(i,:) = randperm(N);
    end

    population = newPopulation;

end

```

UWAGA!

Powyższe fragmenty kodu:

- mają charakter **szkieletów implementacyjnych**,
- mogą wymagać uzupełnienia,
- nie stanowią jedynej poprawnej realizacji algorytmu,
- mają pokazać **kierunek implementacji a nie implementację samą w sobie**.